

RV32I Processor

Three-stage pipeline

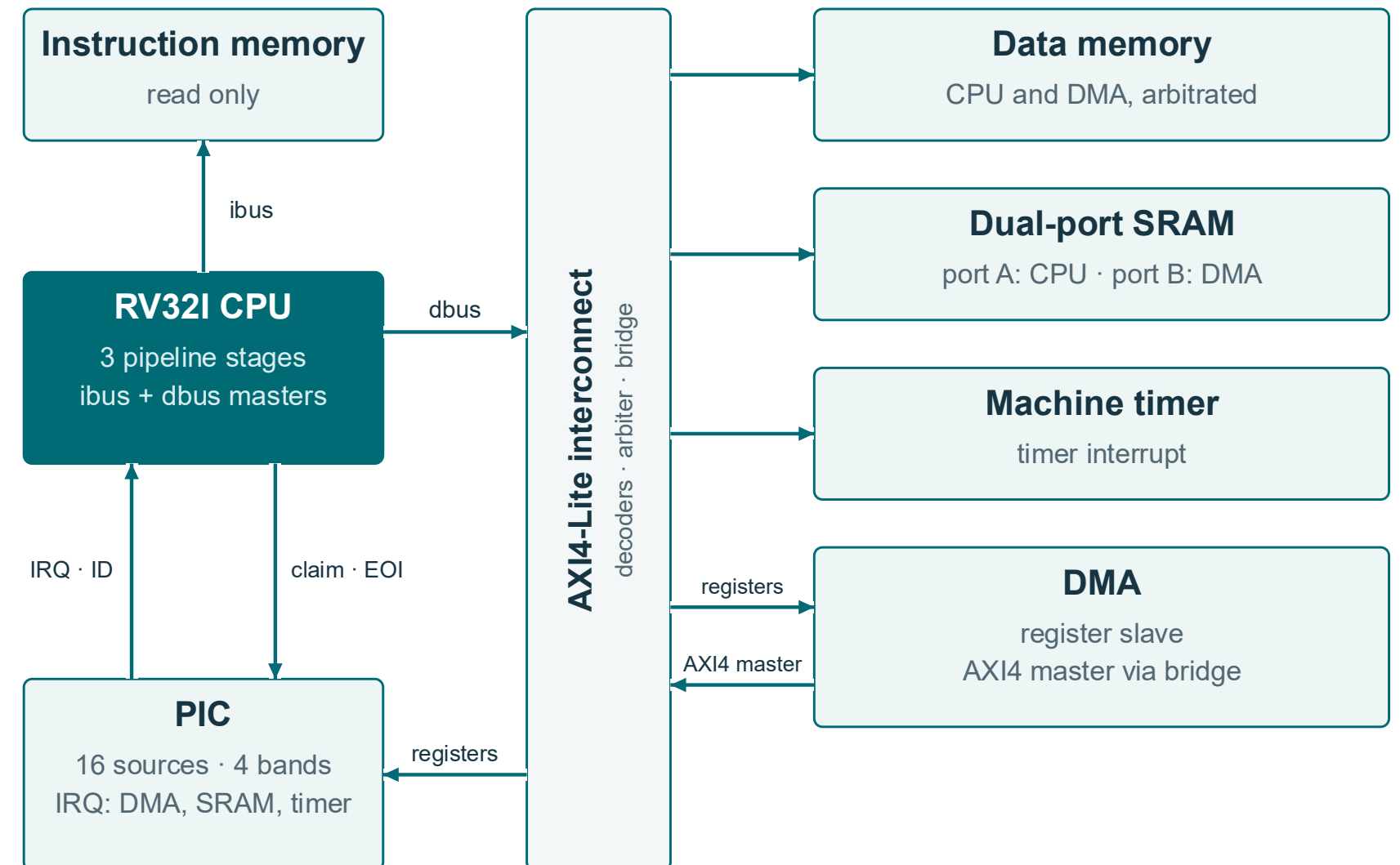
Bunea Cosmin-Andrei

Computer Engineering, year III

Academic year 2025–2026

Transilvania University of Braşov

RTL implementation · interfaces · verification



The task: a three-stage RV32I core with two AXI4-Lite ports

01 RV32I base ISA; no multiply or divide

02 Three stages: fetch, decode + execute, writeback

03 32×32 -bit register file; x0 reads as zero

04 Branch predictor with one direction bit

05 Exceptions and external interrupts

06 CSRs for traps and interrupt control

07 Two independent AXI4-Lite masters

08 Central stall and flush control

09 Interrupt link to the PIC

10 Interrupts taken only at instruction boundaries

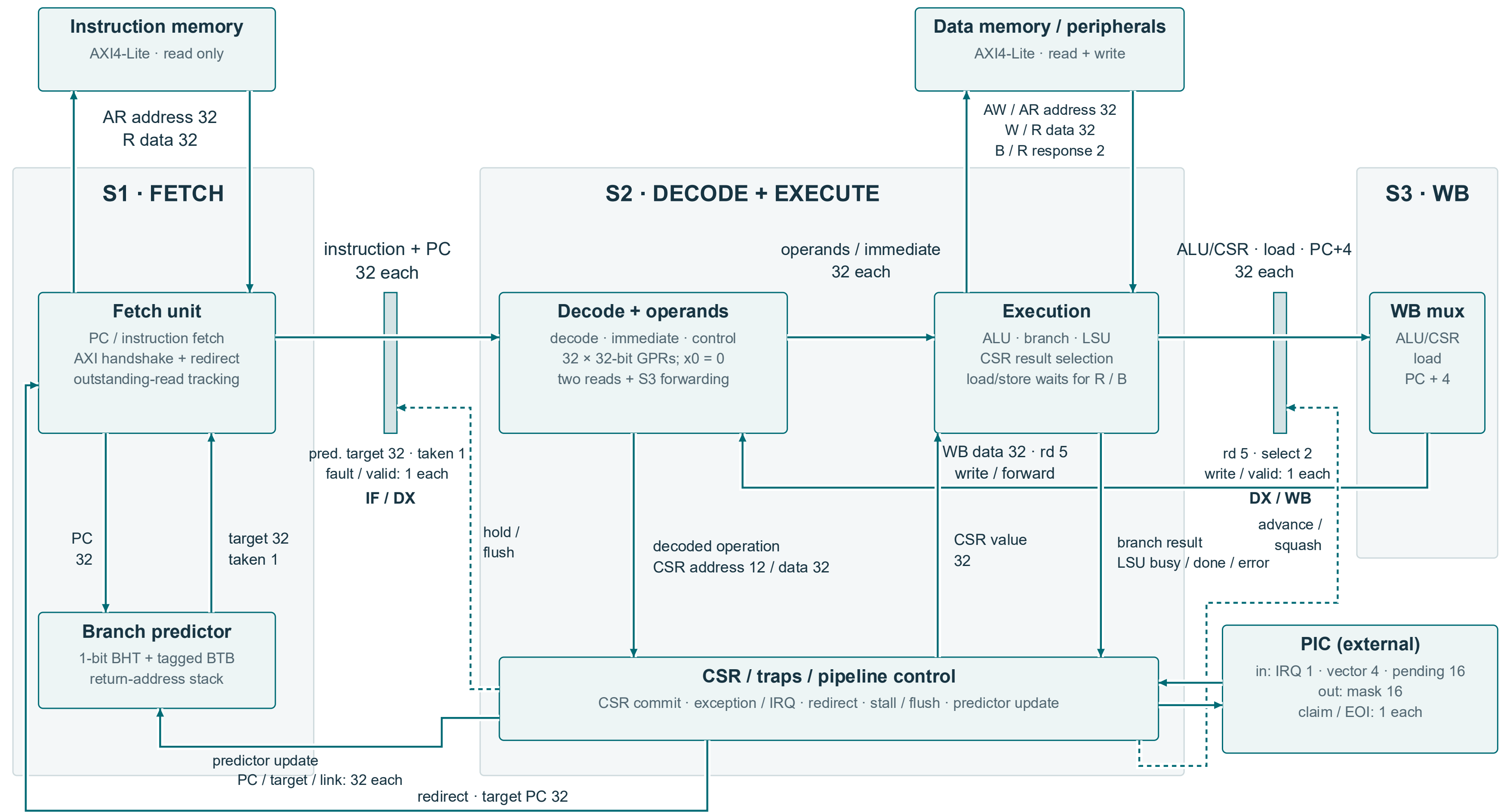
The brief left 23 questions open; five of them shaped the design

Question from the brief

What I decided

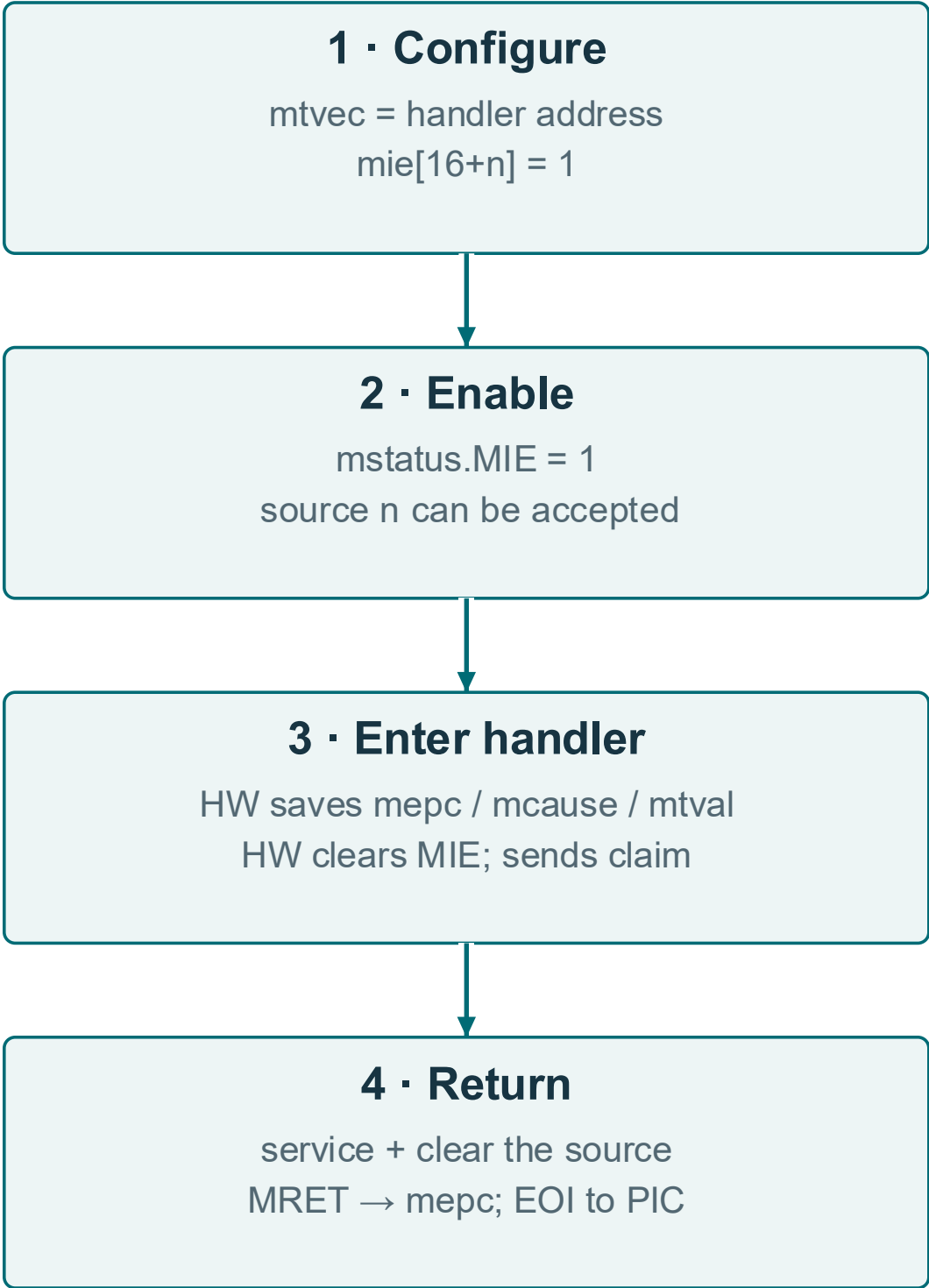
- | | | | |
|---|---|---|--|
| 1 | How large is the predictor, and how is a branch mapped? | → | 128 entries, direct-mapped by PC, with a tag; a miss predicts not taken |
| 2 | Trap, MRET and misprediction in the same cycle? | → | One redirect, only when S2 completes: IRQ, then exception, then MRET, then branch |
| 3 | What happens on an AXI error response? | → | Access-fault exception (cause 1, 5 or 7); the destination register is not written |
| 4 | During a stall, how is a wrong fetch avoided? | → | S1 holds one instruction until S2 is free; a redirect waits for S2, and a wrong-path response is drained |
| 5 | How is a new trap blocked while a handler runs? | → | Trap entry clears MIE; a 16-level stack tells MRET whether to send an EOI |

Three stages; S2 decides when an instruction is complete



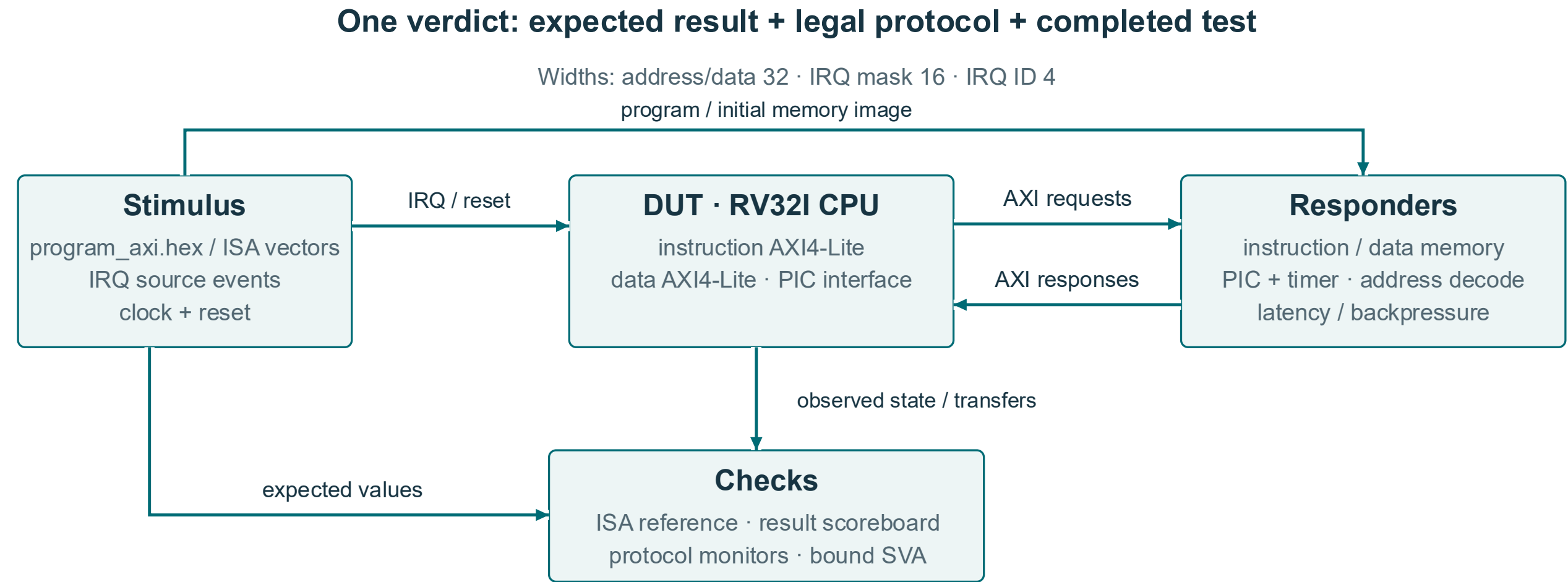
CSR bits 16 to 31 connect software to the PIC

CSR / event	PIC signal	Rule
mie[16+n]	cpu_mask_i[n]	Masked sources are skipped by the PIC
mip[16+n]	pending_o[n]	Every pending source is visible
mcause	cpu_irq_vec_o	$0x8000_0000 + 16 + n$
mtvec, MODE = 1	—	Handler at $\text{BASE} + 4 \times (16 + n)$
Trap entry	cpu_irq_ack_i	One-cycle claim
MRET	cpu_irq_eoi_i	EOI only after an interrupt



CPU instructions ↔ hardware state

The environment checks results, bus protocol and its own tests



Independent model

49,149 instructions in 11 programs,
compared step by step

Latency and backpressure

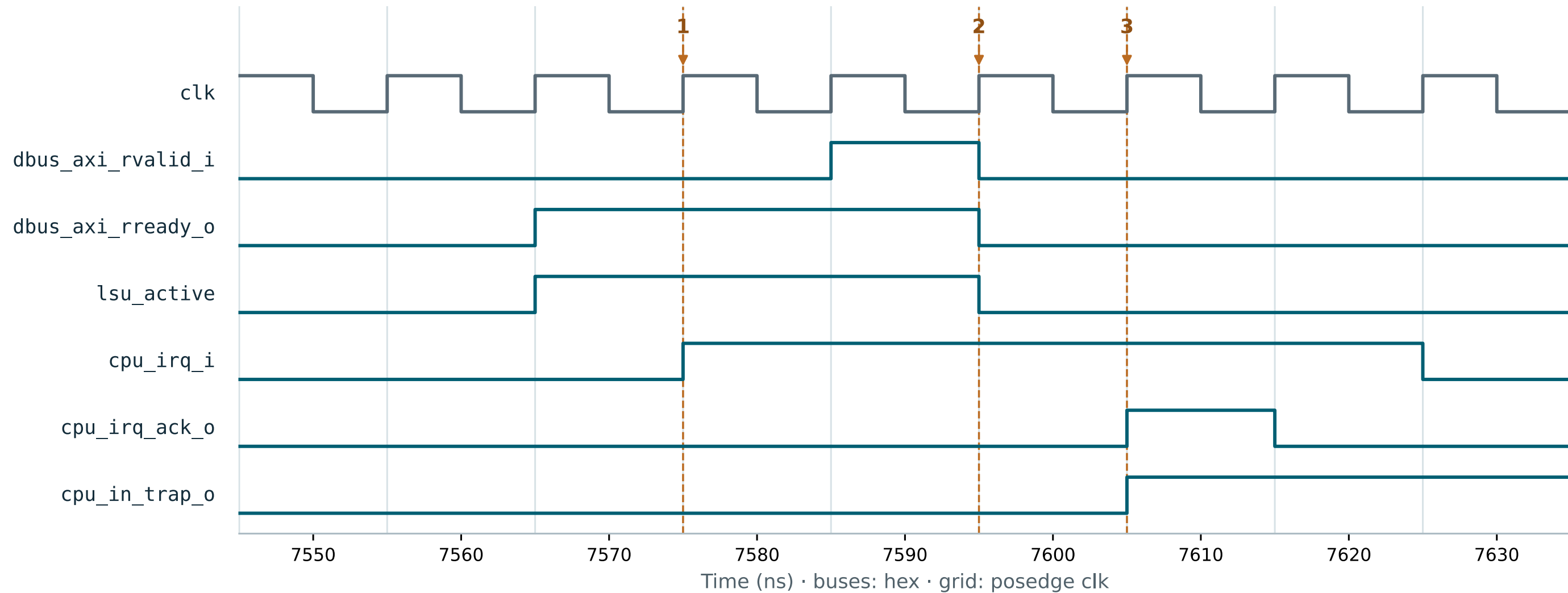
Memories delay replies and stall channels

Mutation suite

25 injected RTL defects; each makes a
named bench fail

ModelSim: 39 CPU/PIC runs · Verilator: 17 benches, 53 runs with assertions bound to the RTL

The IRQ waits for the load to finish before the claim



1 · New IRQ

LSU busy → claim = 0

2 · R handshake

Load complete → S2 may advance

3 · Claim

Trap entry; one-cycle pulse

The markers follow the same instruction and the same IRQ request.

Result: an integrated core, checked instruction by instruction

WHAT I BUILT

1. RV32I pipeline with predictor, forwarding and traps
2. CPU–PIC link: mask, pending, claim and EOI
3. Checks: ISA model, bound assertions, mutation suite

WHAT I LEARNED

- Simultaneous events need a written order; the brief only lists them.
- An accepted AXI transfer cannot be cancelled, and that decides when a trap can start.
- A test counts only if a real defect makes it fail.

Internship: brief → specification → RTL → verification, with a review at each step.